Tutorial Sheet – Exceptions and XML (Revision)

Programming Fundamentals 1
Topic: Exceptions and XML in Java
Purpose: Consolidation and revision before progressing to larger programs

---

Part A: Understanding Exceptions

1. What Are Exceptions?

Answer the following questions:

1. What is an exception in Java?
2. Give one example of a runtime exception.
3. Why is exception handling important in real-world programs?

---

2. Predicting Output (Try–Catch)

Consider the following code:

```
try {
   int[] numbers = {10, 20, 30};
   System.out.println(numbers[3]);
} catch (ArrayIndexOutOfBoundsException e) {
   System.out.println("Index error");
}
```

Answer the following:

a. Does the program crash?

b. What is printed to the console?

---

3. Identifying the Exception

For each of the following scenarios, state **which exception** would occur:

1. Dividing a number by zero
2. Accessing a file that does not exist
3. Trying to access an element outside the bounds of an array

---

4. Writing a Try–Catch Block

Write Java code that:

1. Asks the user to enter two integers
2. Divides the first number by the second
3. Uses a try–catch block to handle division by zero

---

5. Handling Multiple Exceptions in One Try Block

Consider the following scenario: A program asks the user to enter an integer and then accesses an element in an array using that number as an index.

1. Name two different exceptions that could occur in this situation.
2. Write Java code that uses one try block and multiple catch blocks to handle these exceptions.
3. Explain why handling multiple exceptions improves program robustness.

Part B: Working with XML

6. Understanding XML Structure

Consider the following XML snippet:

```
<student>
  <name>Alice</name>
  <course>Computing</course>
</student>
```

Answer the following:

1. What is the root element?
2. Name two child elements.
3. Why is XML useful for storing data?

---

7. XML Syntax Rules

State whether each statement is **true or false**:

1. XML tags are case-sensitive.
2. Every opening tag must have a closing tag.
3. XML is used mainly for program logic.

---

8. Reading Data from XML (Conceptual)

Answer the following:

1. Why might a Java program read data from an XML file?
2. Name one advantage of storing data in XML instead of hard-coding it.

Part C: CRUD Operations with XML

8. Understanding CRUD

Answer the following:

1. What does CRUD stand for?
2. Give one example of a CRUD operation in the context of an XML file.

---

9. Matching CRUD Operations

Match each operation with its description:

- Create
- Read
- Update
- Delete

Descriptions:

a. Removing an element from an XML file
b. Adding a new element to an XML file
c. Changing the value of an existing element
d. Displaying data stored in an XML file

---

10. Thinking in Steps (Algorithm)

Describe, in **plain English**, the steps a program would take to:

- Read an XML file
- Display all student names stored in the file

---

11. Error Handling with Files

Answer the following:

1. Why is file handling often placed inside a try–catch block?
2. What might happen if file-related exceptions are not handled?

---

12. Thinking Challenge

Answer the following:

1. Why might a program use **both** exceptions and XML together?
2. Give one example of a program that could benefit from this combination.
